Bloque 5. Eventos e disparadores

1. Automatización de tarefas nunha base de datos

Os sistemas xestores de bases de datos permiten automatizar operacións que se executan sen necesidade de que un usuario teña que iniciar manualmente cada acción. En MariaDB existen dous mecanismos especialmente útiles para este propósito: os **eventos programados** e os **disparadores ou triggers**.

Aínda que ambos permiten executar instrucións SQL de forma automática, funcionan de maneira diferente:

- **Eventos:** executan instrucións nun momento determinado ou de forma periódica.
- **Disparadores:** executan instrucións automaticamente cando se produce unha operación sobre unha táboa.

Exemplos de automatización

- Eliminar rexistros temporais que teñan máis de 30 días cada noite.
- Actualizar diariamente unha táboa coas estatísticas da base de datos.
- Rexistrar nunha táboa de auditoría cada modificación realizada sobre un produto.
- Impedir que se introduza unha cantidade negativa no stock dun artigo.
- Gardar automaticamente o valor anterior e o novo dunha columna cando se modifica un rexistro.

2. Eventos programados en MariaDB

2.1. Concepto de evento

Un evento é un obxecto da base de datos que permite programar a execución automática dunha ou varias instrucións SQL. A súa execución depende dunha planificación temporal e non dunha operación realizada sobre unha táboa.

Os eventos poden executarse:

- Unha única vez nunha data e hora determinadas.
- De maneira periódica, cada certo intervalo de tempo.
- Durante un período delimitado por unha data de inicio e outra de finalización.

Por exemplo, podemos crear un evento para eliminar os rexistros antigos dunha táboa todos os días ás dúas da madrugada.

2.2. O planificador de eventos: event_scheduler

Para que MariaDB execute os eventos programados, debe estar activo o planificador de eventos, identificado pola variable `event_scheduler`.

Podemos consultar o seu estado mediante:

```
SHOW VARIABLES LIKE 'event_scheduler';
```

O resultado pode indicar os seguintes valores:

Valor	Significado
ON	O planificador está activado e executa os eventos.
OFF	O planificador está desactivado e non executa os eventos.
DISABLED	O planificador está deshabilitado na configuración do servidor.

Para activalo temporalmente, se temos os permisos necesarios, podemos executar:

```
SET GLOBAL event_scheduler = ON;
```

Esta modificación afecta á execución actual do servidor. Para conservar a configuración despois dun reinicio, é necesario establecer a opción correspondente na configuración de MariaDB.

> **Importante:** se o planificador está desactivado, os eventos poden existir na base de datos, pero non se executarán automaticamente.

2.3. Permisos necesarios

A creación e administración de eventos require os permisos correspondentes. O privilexio `EVENT` permite xestionar eventos nunha base de datos.

```
GRANT EVENT ON empresa.* TO 'administrador'@'localhost';
```

Podemos consultar os permisos dun usuario mediante:

```
SHOW GRANTS FOR 'administrador'@'localhost';
```

3. Creación de eventos: CREATE EVENT

3.1. Sintaxe xeral

A instrución `CREATE EVENT` permite crear un evento programado.

```
CREATE EVENT nome_evento
ON SCHEDULE programacion
DO
    instrucions_sql;
```

A programación temporal defínese mediante `ON SCHEDULE`. Os dous mecanismos principais son:

- `AT`: programa unha execución nunha data e hora concretas.
- `EVERY`: programa execucións periódicas cun intervalo determinado.

3.2. Eventos de execución única con AT

Un evento de execución única realízase nun momento determinado. Para iso utilízase `AT`.

```
CREATE EVENT evento_unico
ON SCHEDULE AT '2026-12-01 02:00:00'
DO
    DELETE FROM rexistros_temporais
    WHERE data_creacion < '2026-01-01';
```

Neste exemplo, o evento executará a instrución de eliminación o día 1 de decembro de 2026 ás 02:00 horas.

Por defecto, os eventos de execución única elimínanse unha vez completada a súa execución. Se queremos conservalos, podemos utilizar `ON COMPLETION PRESERVE`.

```
CREATE EVENT evento_conservado
ON SCHEDULE AT '2026-12-01 02:00:00'
ON COMPLETION PRESERVE
DO
    INSERT INTO rexistro_tarefas(descricion, data_execucion)
    VALUES ('Execución do evento', NOW());
```

3.3. Eventos periódicos con EVERY

Se queremos executar unha tarefa repetidamente, utilizamos `EVERY`, indicando o intervalo de tempo.

```
CREATE EVENT evento_diario
ON SCHEDULE EVERY 1 DAY
DO
    DELETE FROM rexistros_temporais
    WHERE data_creacion < DATE_SUB(NOW(), INTERVAL 30 DAY);
```

Este evento elimina diariamente os rexistros temporais que teñan máis de 30 días.

Os intervalos poden expresarse en diferentes unidades:

Unidade	Exemplo
SECOND	`EVERY 30 SECOND`
MINUTE	`EVERY 15 MINUTE`
HOUR	`EVERY 2 HOUR`
DAY	`EVERY 1 DAY`
WEEK	`EVERY 1 WEEK`
MONTH	`EVERY 1 MONTH`
YEAR	`EVERY 1 YEAR`

3.4. Establecer inicio e finalización

Podemos indicar cando debe comezar a execución periódica mediante `STARTS` e cando debe rematar mediante `ENDS`.

```
CREATE EVENT evento_temporal
ON SCHEDULE EVERY 1 DAY
STARTS '2026-12-01 01:00:00'
ENDS '2026-12-31 01:00:00'
DO
    INSERT INTO rexistro_tarefas(descricion, data_execucion)
    VALUES ('Tarefa periódica', NOW());
```

Neste caso, o evento executarase diariamente a partir do 1 de decembro e deixará de facelo despois da data de finalización establecida.

> **Recorda:** `AT` utilízase para unha execución puntual, mentres que `EVERY` permite establecer unha programación recorrente.

4. Administración dos eventos

4.1. Consultar os eventos existentes

Podemos consultar os eventos dunha base de datos mediante:

```
SHOW EVENTS;
```

Para consultar a definición completa dun evento concreto:

```
SHOW CREATE EVENT evento_diario;
```

Estas instrucións permiten comprobar a programación, o estado e as instrucións SQL asociadas a cada evento.

4.2. Modificar un evento

A instrución `ALTER EVENT` permite modificar a configuración dun evento existente.

```
ALTER EVENT evento_diario
ON SCHEDULE EVERY 12 HOUR;
```

Neste exemplo, modifícase a frecuencia de execución para que se realice cada doce horas.

4.3. Activar e desactivar eventos

Podemos activar ou desactivar temporalmente un evento sen necesidade de eliminalo.

```
ALTER EVENT evento_diario DISABLE;
```

Para volver activalo:

```
ALTER EVENT evento_diario ENABLE;
```

Un evento desactivado continúa existindo, pero non se executa mentres permaneza nese estado.

4.4. Eliminar eventos

Para eliminar definitivamente un evento utilízase `DROP EVENT`.

```
DROP EVENT evento_diario;
```

Se queremos evitar un erro cando o evento non existe, podemos utilizar `IF EXISTS`.

```
DROP EVENT IF EXISTS evento_diario;
```

5. Disparadores ou triggers

5.1. Concepto de disparador

Un disparador, ou *trigger*, é un obxecto da base de datos que contén instrucións SQL que se executan automaticamente cando se produce unha operación determinada sobre unha táboa.

A diferenza dos eventos, os disparadores non dependen dunha programación temporal, senón dunha modificación ou inserción de datos.

As operacións que poden activar un disparador son:

- `INSERT`: cando se insire un rexistro.
- `UPDATE`: cando se modifica un rexistro.

- `DELETE`: cando se elimina un rexistro.

5.2. Momentos de execución: BEFORE e AFTER

Un disparador pode executarse antes ou despois da operación que o activa.

Tipo	Funcionamento	Aplicación habitual
`BEFORE`	Execútase antes de realizar a operación.	Validar ou modificar os datos antes de gardalos.
`AFTER`	Execútase despois de realizar a operación.	Rexistrar cambios ou actualizar información relacionada.

Por exemplo:

- `BEFORE INSERT`: comprobar que os datos dun novo produto son correctos.
- `AFTER INSERT`: rexistrar a alta dun produto nunha táboa de auditoría.
- `BEFORE UPDATE`: comprobar que o novo prezo non é negativo.
- `AFTER DELETE`: gardar información do rexistro eliminado.

5.3. Disparadores por fila

En MariaDB, os triggers execútanse por cada fila afectada pola operación. Para indicalo, a súa definición inclúe obrigatoriamente `FOR EACH ROW`.

Por exemplo, se unha instrución `UPDATE` modifica dez rexistros, o disparador correspondente executarase dez veces, unha por cada fila modificada.

6. Creación de disparadores: CREATE TRIGGER

6.1. Sintaxe xeral

A instrución `CREATE TRIGGER` permite crear un disparador asociado a unha táboa.

```
DELIMITER //

CREATE TRIGGER nome_trigger
BEFORE INSERT ON nome_taboa
FOR EACH ROW
BEGIN
    -- Instrucións SQL
END //

DELIMITER ;
```

Na definición debemos indicar:

- O nome do disparador.
- O momento de execución: `BEFORE` ou `AFTER`.
- A operación que o activa: `INSERT`, `UPDATE` ou `DELETE`.
- A táboa sobre a que actúa.
- As instrucións que se executarán automaticamente.

O uso de `DELIMITER` permite que o cliente SQL interprete correctamente os bloques que conteñen varias instrucións. É unha directiva do cliente, non unha instrución SQL executada polo servidor.

6.2. Combinacións posibles

Podemos crear disparadores para as seguintes combinacións:

Operación	Antes	Despois
INSERT	BEFORE INSERT	AFTER INSERT
UPDATE	BEFORE UPDATE	AFTER UPDATE
DELETE	BEFORE DELETE	AFTER DELETE

Por exemplo, un disparador que se execute despois dunha eliminación:

```
DELIMITER //

CREATE TRIGGER rexistrar_borrado
AFTER DELETE ON produtos
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_produtos
        (id_produto, nome, data_operacion)
    VALUES
        (OLD.id, OLD.nome, NOW());
END //

DELIMITER ;
```

Cando se elimina un produto, o disparador garda automaticamente o identificador, o nome e a data da operación na táboa de auditoría.

7. Uso de OLD e NEW

7.1. Concepto

Dentro dun disparador podemos acceder aos valores dos rexistros afectados mediante as palabras clave `OLD` e `NEW`.

- `OLD`: representa os valores que tiña a fila antes da operación.
- `NEW`: representa os valores que terá a fila despois da operación.

A súa dispoñibilidade depende da operación que active o disparador.

Operación	OLD	NEW
INSERT	Non dispoñible	Si
UPDATE	Si	Si
DELETE	Si	Non dispoñible

7.2. Acceso aos valores con NEW

Nun disparador asociado a unha inserción podemos consultar os valores do novo rexistro mediante `NEW`.

```
DELIMITER //

CREATE TRIGGER validar_stock
BEFORE INSERT ON produtos
FOR EACH ROW
BEGIN
    IF NEW.stock < 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'O stock non pode ser negativo';
    END IF;
END //

DELIMITER ;
```

O disparador comproba o valor do stock antes de inserir o produto. Se é negativo, xera un erro e impide que a operación continúe.

7.3. Modificación de valores con NEW

Nos disparadores `BEFORE INSERT` e `BEFORE UPDATE` podemos modificar os valores de `NEW`.

```
DELIMITER //

CREATE TRIGGER normalizar_nome
BEFORE INSERT ON produtos
FOR EACH ROW
BEGIN
    SET NEW.nome = UPPER(NEW.nome);
END //

DELIMITER ;
```

Neste caso, o nome do produto gárdase en maiúsculas, independentemente de como o introducise o usuario.

> **Importante:** non se pode modificar `NEW` nun disparador `AFTER`. A modificación dos valores novos debe realizarse antes de que se complete a operación.

7.4. Comparación de valores con OLD e NEW

Nunha operación `UPDATE` podemos comparar os valores anteriores e posteriores á modificación.

```
DELIMITER //

CREATE TRIGGER rexistrar_cambio_prezo
AFTER UPDATE ON produtos
FOR EACH ROW
BEGIN
    IF OLD.prezo <> NEW.prezo THEN
        INSERT INTO auditoria_prezos
            (id_produto, prezo_anterior, prezo_novo, data_cambio)
        VALUES
            (OLD.id, OLD.prezo, NEW.prezo, NOW());
    END IF;
END //

DELIMITER ;
```

Este disparador comproba se o prezo cambiou. Se é así, almacena o prezo anterior e o novo nunha táboa de auditoría.

8. Aplicacións dos disparadores

8.1. Validación de datos

Os disparadores permiten establecer regras adicionais antes de inserir ou modificar información.

Podemos utilizar `BEFORE INSERT` ou `BEFORE UPDATE` para comprobar condicións e impedir operacións incorrectas.

```
DELIMITER //

CREATE TRIGGER validar_prezo
BEFORE UPDATE ON produtos
FOR EACH ROW
BEGIN
    IF NEW.prezo <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'O prezo debe ser maior que cero';
    END IF;
END //

DELIMITER ;
```

8.2. Rexistro de cambios e auditoría

Outra aplicación habitual é gardar un historial das modificacións realizadas sobre unha táboa.

Para iso créase unha táboa auxiliar que almacena información como:

- Identificador do rexistro modificado.
- Valores anteriores e posteriores.
- Tipo de operación realizada.
- Data e hora do cambio.

Por exemplo, podemos crear unha táboa para almacenar os cambios de prezo:

```
CREATE TABLE auditoria_prezos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_produto INT,
    prezo_anterior DECIMAL(10,2),
    prezo_novo DECIMAL(10,2),
    data_cambio DATETIME
);
```

O disparador `AFTER UPDATE` presentado anteriormente encargarase de inserir os cambios nesta táboa.

8.3. Actualización automática de información relacionada

Os disparadores tamén poden utilizarse para actualizar información auxiliar cando se modifica unha táboa.

Por exemplo, cando se insire unha nova liña nunha táboa de vendas, un disparador pode actualizar o stock do produto correspondente. Este tipo de automatización debe deseñarse con coidado para evitar actualizacións duplicadas ou inconsistencias.

9. Administración e eliminación de disparadores

9.1. Consultar os disparadores existentes

Para consultar os disparadores dunha base de datos:

```
SHOW TRIGGERS;
```

Para consultar a definición dun disparador concreto:

```
SHOW CREATE TRIGGER rexistrar_cambio_prezo;
```

9.2. Eliminar un disparador

Para eliminar un disparador utilizamos `DROP TRIGGER`.

```
DROP TRIGGER rexistrar_cambio_prezo;
```

Podemos evitar erros se o disparador non existe utilizando:

```
DROP TRIGGER IF EXISTS rexistrar_cambio_prezo;
```

> **Diferenza importante:** os eventos poden activarse ou desactivarse mediante `ALTER EVENT`. Os disparadores non dispoñen dun mecanismo equivalente de activación e desactivación: para deixar de utilizalos é necesario eliminalos ou modificar a súa definición.

10. Diferenzas entre eventos e disparadores

Característica	Eventos	Disparadores
Activación	Por tempo ou calendario.	Por operación sobre unha táboa.
Operacións	Execución de instrucións SQL programadas.	INSERT, UPDATE e DELETE.
Execución	Nunha data ou de forma periódica.	Por cada fila afectada.
Activación xeral	Depende de event_scheduler.	Non depende do planificador de eventos.
Uso habitual	Limpeza e mantemento automático.	Validación, auditoría e actualización de datos.

11. Boas prácticas e consideracións

- Utilizar nomes descritivos para identificar eventos e disparadores.
- Comprobar que o planificador de eventos está activo antes de probar unha programación.
- Definir correctamente as datas e os intervalos dos eventos.
- Evitar executar operacións innecesarias en eventos demasiado frecuentes.
- Manter os disparadores sinxelos e realizar neles unicamente as operacións necesarias.
- Ter en conta que un disparador pode executarse varias veces se unha operación afecta a varias filas.
- Comprobar os efectos dos triggers sobre a integridade e o rendemento da base de datos.
- Documentar as tarefas automatizadas para facilitar o mantemento.
- Ter en conta os permisos do usuario que define e executa os eventos e disparadores.

> **Precaución:** os disparadores forman parte da execución da operación que os activa. Non se debe introducir neles lóxica que realice operacións incompatibles co contexto do trigger, como confirmar ou desfacer transaccións mediante `COMMIT` ou `ROLLBACK` .

12. Exercicios prácticos

Os seguintes exercicios permiten practicar a creación e administración de eventos e disparadores nunha base de datos MariaDB.

Exercicio 1. Eventos programados

1. Consulta o estado da variable `event_scheduler` .
2. Activa o planificador de eventos se está desactivado.
3. Crea un evento que insira un rexistro nunha táboa de rexistro nunha data e hora determinadas.
4. Crea outro evento que se execute diariamente e realice unha tarefa de mantemento.
5. Consulta os eventos creados e comproba as súas definicións.
6. Modifica a programación dun dos eventos.
7. Desactiva o evento, volve activalo e, finalmente, elimínao.

Exercicio 2. Validación mediante triggers

1. Crea unha táboa de produtos con identificador, nome, prezo e stock.
2. Define un disparador que impida inserir produtos cun prezo ou stock negativo.

3. Define outro disparador que normalice os nomes dos produtos antes de gardalos.
4. Realiza inserciónss válidas e non válidas para comprobar o funcionamento dos disparadores.
5. Comproba os resultados almacenados na táboa.

Exercicio 3. Auditoría de modificacións

1. Crea unha táboa auxiliar para almacenar o historial de cambios dos produtos.
2. Define un disparador que rexistre os cambios de prezo realizados mediante `UPDATE`.
3. Realiza varias modificacións sobre os produtos.
4. Consulta o historial e comproba que se gardan os valores anteriores e posteriores.
5. Define un disparador que rexistre a eliminación dun produto.
6. Comproba que os datos eliminados quedan almacenados na táboa de auditoría.

Exercicio 4. Automatización integrada

Deseña unha solución que combine eventos e disparadores para automatizar o mantemento dunha base de datos.

1. Utiliza disparadores para validar as modificacións dunha táboa.
2. Garda un historial das operacións relevantes.
3. Programa un evento periódico para realizar unha tarefa de mantemento sobre os rexistros antigos.
4. Comproba o funcionamento dos mecanismos de automatización.
5. Documenta as solucións implementadas e os resultados obtidos.

13. Resumo do bloque

- Os **eventos** permiten executar instrucións SQL nun momento determinado ou de maneira periódica.
- O planificador `event_scheduler` debe estar activo para que MariaDB execute os eventos.
- `CREATE EVENT` crea eventos e `ON SCHEDULE` define a súa programación.
- `AT` programa unha execución puntual e `EVERY` establece unha execución recorrente.
- `ALTER EVENT` permite modificar, activar e desactivar eventos.
- Os **disparadores** executan instrucións automaticamente cando se produce un INSERT, UPDATE ou DELETE.
- `BEFORE` executa o trigger antes da operación e `AFTER` despois.
- `OLD` permite consultar os valores anteriores e `NEW` os valores novos.

- Os disparadores poden utilizarse para validar datos, rexistrar cambios e automatizar operacións relacionadas.
- `SHOW EVENTS` e `SHOW TRIGGERS` permiten consultar os elementos de automatización existentes.